

RBE4701-C25: AI for Robotics

Professor Pinciroli

Project 2: Bomberman Escape Mode with Q-Learning

Group 7: Chad Nguyen, Katarzyna (Kasia) Racka, William Sobral

Approach - Summarized

Our approach consists of using a state machine with the TRAVELING, FLEEING, BOMBING, and WAITING states.

The TRAVELING state uses A* to navigate the character towards the exit cell, and to place a bomb when sufficiently close to a wall that blocks its path. After placing the bomb, the character enters the BOMBING state, where it checks if it is in a safe location. If it is not, it moves to a safe location, out of the explosion range of the bomb. If that safe location has no monster in the vicinity, the character enters the WAITING state. The character could also enter the waiting state from the TRAVELING state if a bomb exists. As a catch for safety, if the character is still in range of the bomb while in the WAITING state, it randomly moves to a safe location. Otherwise, the character stays still.

If the character detects a monster nearby while in any of the above states, it begins training through Approximate Q-Learning so that it can determine the best action. Hand-created features are utilized to calculate each Q-value and train the weights for Q-Learning. Training occurs in multiple iterations to explore several games for weight training. At the start of each iteration, weights are normalized to prevent them from blowing up out of control. Then, Q-Learning uses SensedWorld to learn about and experiment with different states and until death or successful exit of the character. Based on the feature weights after training, the character chooses the best next action. The character enters the FLEEING state, where it then uses the calculated weights to determine its next best action at each state. At this point, if there is no longer a monster nearby or any bombs, the character resumes the TRAVELING state. If there is no

monster nearby but there is a bomb, the character moves to a safe location before entering the WAITING state.

Approach - Specifics

Usage of SensedWorld

Each iteration of training through Q-Learning used SensedWorld to evaluate weights. In this way, the dummy character could experiment with different actions and different feature states without the actual game character dying. The next step of the sensed world is also used to evaluate the Q value of each next state based on a specific action, allowing the algorithm to extract the max Q value and hence the reward associated with the present state. Then, based on the weights, the dummy character picks the best action before continuing with training.

Reward Evaluation

The reward associated with each state is based on things in the world and the coordinate location of the dummy character. Locations that would result in death (e.g. standing on a monster) are penalized ($r = -5000$) while reaching the exit is rewarded ($r = +5000$). To create a sense of urgency for the character to proceed to the exit, the reward of a standard cell is -1.

Feature Evaluation

In total, the amount of features calculated for each world state is 10 (currently) features. All of the features chosen give the character an opportunity to learn why it succeeded or died, allowing it to incentivize or penalize certain game states. The 1st feature is based on the average monster distance squared to incentivize being farther away from the monsters. The 2nd feature is based on the amount of monsters in the world to prioritize removing monsters from the board. The 3rd feature is based on the length of the exit path to prioritize getting closer to the exit. The 4th and 5th feature is based on how close the character is to its bomb and if it is in the blast radius. These features allow the bomberman to learn about the risk of being close to the bomb. The 6th feature is based on the closest monster to penalize being in danger. The 7th feature is based on how

many actions are available for the character to choose from to allow the character to penalize trapping itself in corners of the map. The 8th feature is based on the angle between the monster and the character's next step. Using the law of cosines, this allows the character to learn if the monster is far away or in its path to the exit. Finally, the 9th and 10th features are based on the characters horizontal and vertical distance to the closest monster to penalize the character further for moving closer to danger.

Approximate Q-Learning

Picking the Next Best Action

In selecting the best action to take, the character ignores actions to cells it cannot go to. Then, it examines all possible actions and evaluates the Q value of the new state, using the features in SensedWorld. The character then chooses the action that would result in the highest Q value, and executes it.

While the character is training its feature weights, it considers walking into danger as a possible action. This allows the algorithm to return a negative reward for death, affecting feature weights. However, once the character has concluded training, it does not consider actions that would endanger itself. This keeps the character alive whilst still respecting the evaluation from the weights and features.

Experimental Evaluation

Our character AI performed well against the stupid monster. His performance against the self-preserving monster was a bit worse. The AI performed much worse against the aggressive monster and in variant 5 with both monsters.

Table 1: Character wins in Bomberman Escape Mode					
	Variant 1	Variant 2	Variant 3	Variant 4	Variant 5

1	True	False	True	False	False
2	True	True	False	True	False
3	True	False	False	True	False
4	True	True	False	False	False
5	True	True	True	False	False
6	True	False	False	False	False
7	True	True	True	False	False
8	True	True	False	False	False
9	True	False	False	False	True
10	True	True	True	False	True